

Deep Learning

3. Multi Layer Perceptrons

Romain Tavenard

Limitations of the Perceptron

- Input: $x = \{x_0, \dots, x_D\}$
- Output:
$$a = \varphi\left(\sum_j w_j x_j + b\right)$$
- Can do
 - Linear regression
 - Linear boundaries for classification
 - Single output

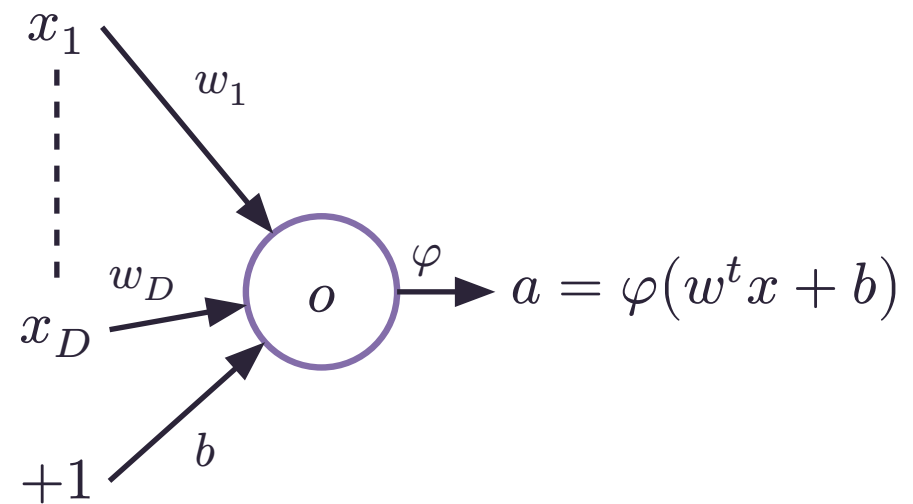
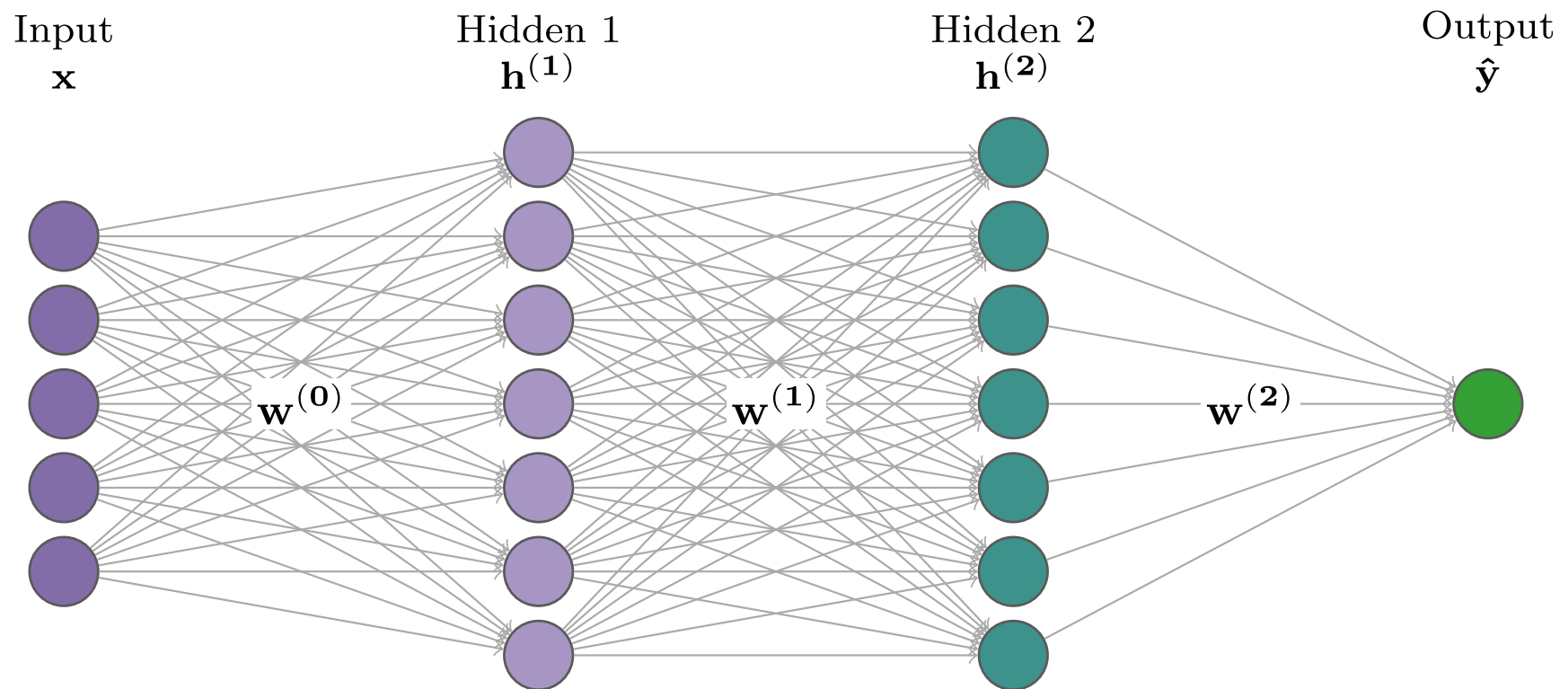
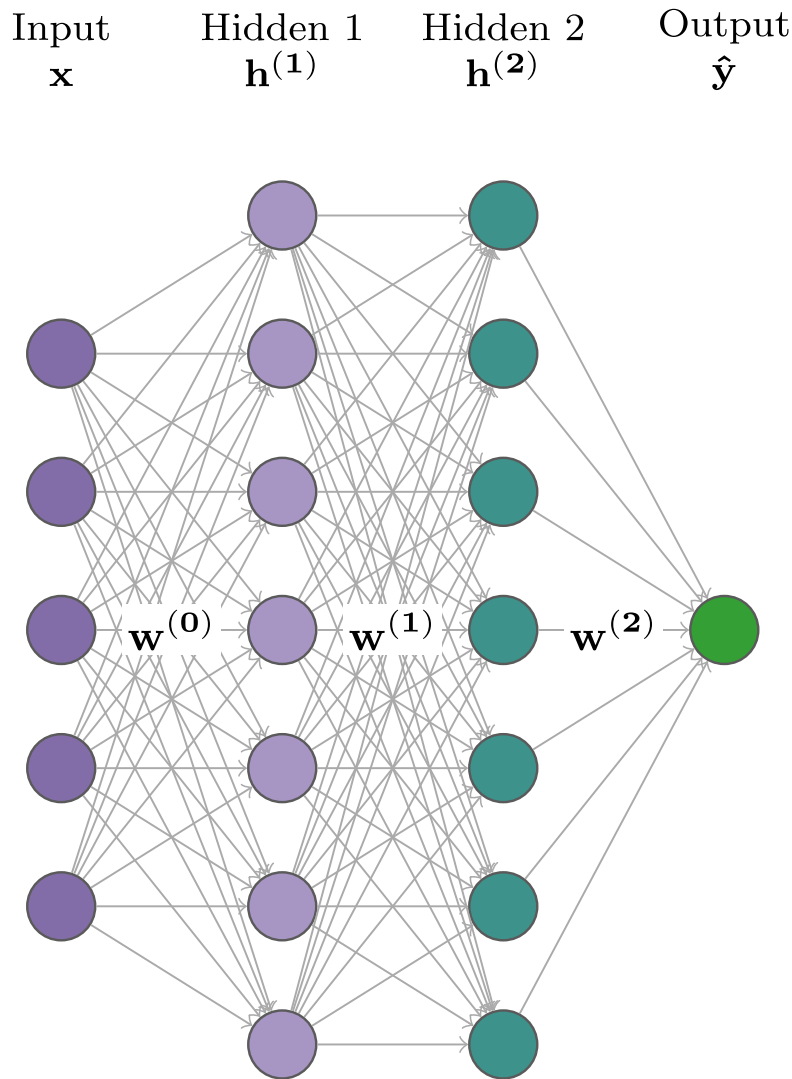


Figure: Perceptron diagram

Multi-Layer Perceptron (MLP) model (Rumelhart, Hinton & Williams, 1985)



Multi-Layer Perceptron (MLP) model (Rumelhart, Hinton & Williams, 1985)



$$\hat{y} = \varphi_{\text{out}} \left(\sum_i w_i^{(2)} h_i^{(2)} + b^{(2)} \right)$$

$$\forall i, h_i^{(2)} = \varphi \left(\sum_j w_{ij}^{(1)} h_j^{(1)} + b_i^{(1)} \right)$$

$$\forall i, h_i^{(1)} = \varphi \left(\sum_j w_{ij}^{(0)} x_j + b_i^{(0)} \right)$$

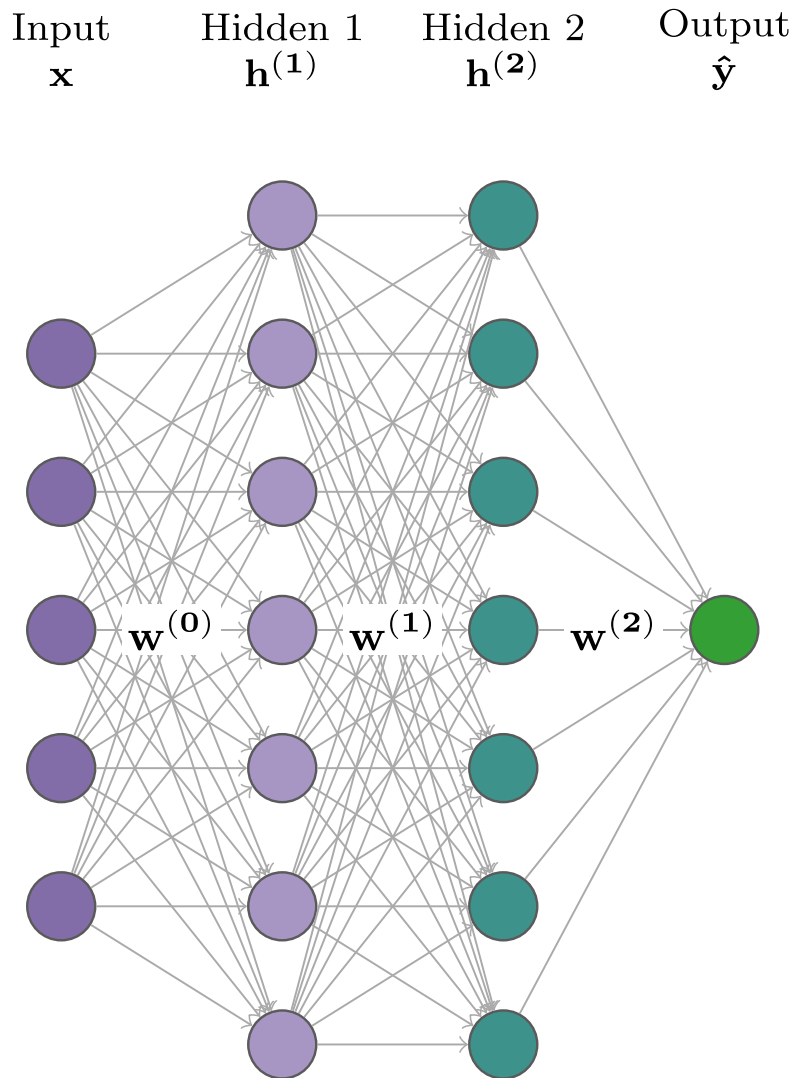
Why introduce hidden layer(s)?

Universal approximation theorem (Cybenko, 1989)

- **Intuition:** hidden layers allow the network to learn non-linear transformations of the input — they reshape the data into a space where the final (linear) output layer can solve the task
- **Theoretical result:** a single hidden layer with enough neurons can approximate **any** reasonable function arbitrarily well
- **Practical takeaway:**
 - one hidden layer is sufficient **in theory**
 - but the required number of neurons may be huge
 - in practice, **stacking more (smaller) layers** is more efficient and generalizes better

Building blocks of neural networks

Input/Output layers



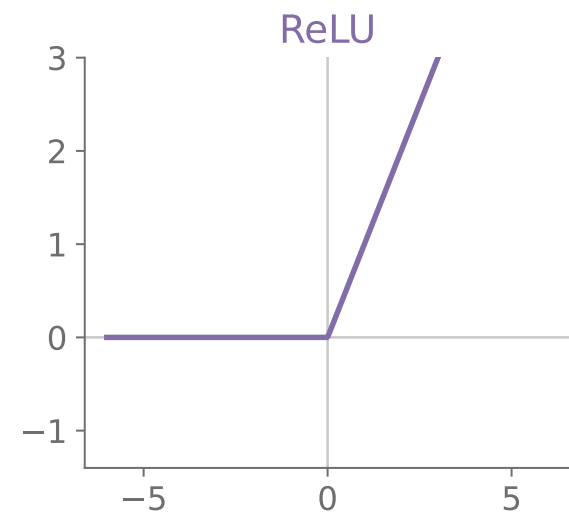
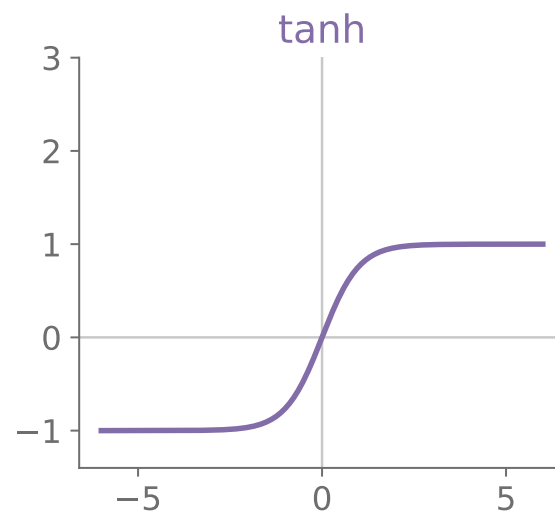
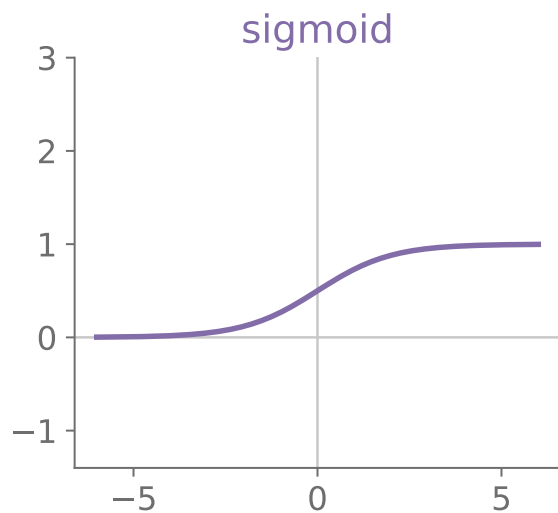
Given a dataset (X, y)

- Constraints on model structure:
 - Input layer dimension is the number of features in X
 - Output layer has as many units as columns in y

Building blocks of neural networks

Activation functions

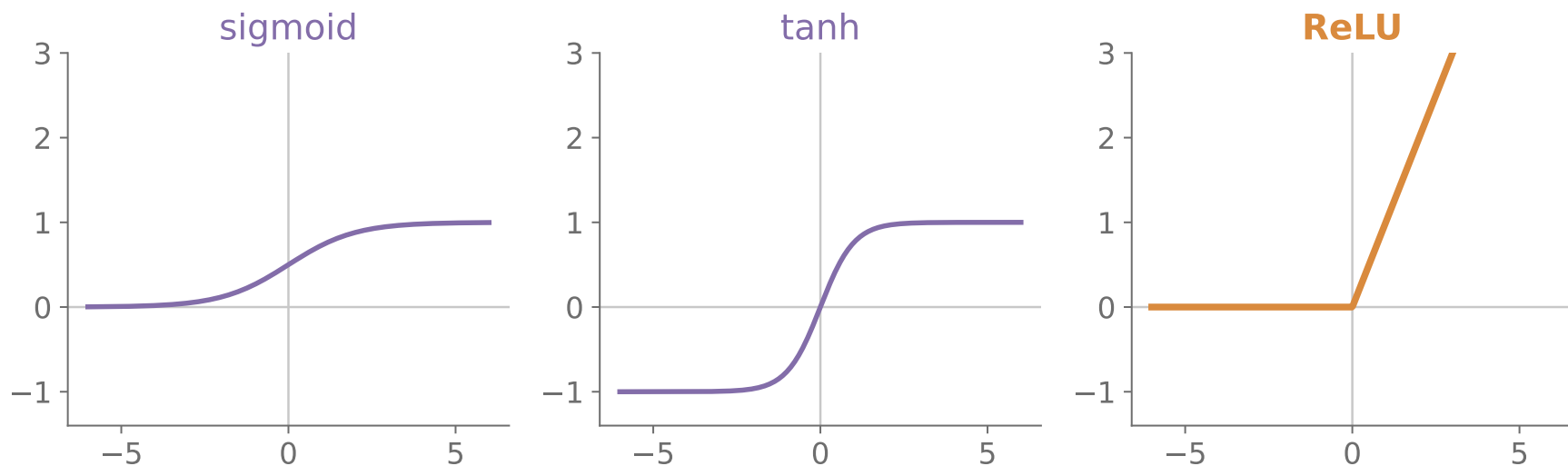
- Important features
 - φ should be differentiable almost everywhere
 - Non-linearities
 - Some linear regime
- Examples



Building blocks of neural networks

Activation functions: the reign of ReLU

- ReLU has become the default choice **for internal layers** over time
- 2 main reasons:
 - cheap to compute (both ReLU and its derivative)
 - vanishing gradients phenomenon (more on that later)



ReLU is now the default for internal layers

Building blocks of neural networks

Activation functions: the case of the output layer

- Output activation functions drive the values:
 - identity (“linear” in keras): any real
 - ReLU: any positive value
 - sigmoid: any value in $[0, 1]$
 - softmax: >0 and sums to 1 (across output neurons)

